CSCI 2406: Topic 04

Integer Arithmetic and Performance Basics

Addition, Subtraction, Multiplication, Division, Shifts, NZCV Flags, and Execution Time Metrics in AArch64

Computer Organization & Architecture | Topic 4

Topic 4 Outline

Topic Objective

Analysis of how arithmetic operations manipulate fixed-width bit patterns, how condition flags govern control flow, and how hardware performance is measured using instruction counts, CPI, and clock rates.

- **Section 1**: Integer Addition, Subtraction, and Unsigned Wraparound
- **Section 2**: Two's Complement Arithmetic and Signed Overflow
- **Section 3**: AArch64 Condition Flags (NZCV) and Flag-Setting Instructions
- **Section 4**: Multiplication, Division, and Logical/Arithmetic Shifts
- **Section 5**: Processor Performance: Clock Rate, CPI, and CPU Time
- **Section 6**: Summary and Worked Self-Test Exercises

Section 01

Integer Addition, Subtraction, and Unsigned Wraparound

Why Integer Arithmetic Matters

Integer arithmetic executes continuously during program execution:

- **Core Operations**: Memory addresses, loop counters, array indexing, and comparisons rely entirely on integer manipulation.
- **Fixed-Width Registers**: AArch64 registers hold fixed-width values (32-bit `w` or 64-bit `x`).
- **Arithmetic Effects**: Operations can wrap around, generate carries, or trigger signed overflows.
- **Control Flow Integration**: Condition flags connect arithmetic outputs directly to subsequent control decisions.

Binary Addition and Unsigned Ranges

Addition Mechanics

- Proceeds from least significant bit (LSB) to most significant bit (MSB).
- $1 + 1$ produces 0 with a carry out of 1.
- Carries propagate across higher-order bit positions.

Unsigned Addition & Carry Out

- Treats all bits strictly as magnitude.
- An n-bit destination keeps only the low-order n bits.
- A carry-out indicates the mathematical result exceeded the range 0 to $2^n - 1$.

Example (32-Bit Unsigned)

$$\text{0xFFFFFFFF} + \text{0x00000001} = \text{0x00000000 with } C = 1$$

Binary Subtraction Using Two's Complement

Hardware avoids dedicated subtraction logic by reusing addition circuitry:

- Subtraction ($A - B$) is computed as $A + (-B)$.
- To form $-B$, invert all bits of B (one's complement) and add 1.
- Any carry generated beyond the register width is discarded from the stored container.

4-Bit Subtraction Example ($6 - 3$)

- $+3 = 0011_2 \rightarrow -3 = \text{invert}(0011) + 1 = 1100 + 1 = 1101_2$.
- Compute $6 + (-3)$: $0110_2 + 1101_2 = 1\,0011_2$.
- Discarding the carry bit yields stored result $0011_2 = +3_{10}$.

Section 02

Two's Complement Arithmetic and Signed Overflow

Signed vs. Unsigned Interpretation

The ALU operates exclusively on raw bit patterns without inherent data types:

- The exact same bit container can be interpreted as either signed or unsigned.
- Signed and unsigned addition execute via the identical underlying binary adder.
- Carry and overflow flags interpret the output differently depending on intended data type.

Example (8-Bit Pattern: $1111\,1111_2$)

- Unsigned interpretation equals 255.
- Signed two's complement interpretation equals -1.

Detecting Signed Overflow

Definition of Signed Overflow

Signed overflow occurs when the true mathematical result of a two's complement addition falls outside the representable signed range.

- **Rule of Thumb**: Adding two positive values must not produce a negative result; adding two negative values must not produce a positive result.
- Hardware records signed overflow in the **V** (Overflow) flag.

8-Bit Overflow Example

$$0111\,1111_2(+127) + 0000\,0001_2(+1) = 1000\,0000_2(-128), \text{ with } V = 1$$

Carry versus Signed Overflow

Carry and overflow evaluate distinct numerical conditions and are not interchangeable:

Carry Flag (C)

- Relates primarily to unsigned arithmetic.
- Records carry out of the most significant bit.
- Example: 0xFF + 0x01 = 0x00, with $C = 1$ (unsigned wrap).

Overflow Flag (V)

- Relates to signed two's complement arithmetic.
- Indicates a sign violation exceeding container bounds.
- Example: 0x7F + 0x01 = 0x80, with $V = 1$ (signed overflow).

Section 03

AArch64 Condition Flags (NZCV) and Flag-Setting Instructions

Processor State and Condition Flags (NZCV)

Arithmetic results automatically update condition flags stored in **PSTATE**:

Flag	Condition Evaluated	Technical Meaning
N (Negative)	Result is negative signed value	Copied from MSB (bit 63 or 31)
Z (Zero)	Result equals exactly zero	Set (1) if all result bits are zero
C (Carry)	Unsigned carry or borrow	Unsigned overflow / carry out from MSB
V (Overflow)	Signed two's complement overflow	Signed range violation detected

Flag Persistence

Once set, the NZCV flag pattern persists across subsequent instructions until modified by another arithmetic or comparison instruction.

Flag-Setting Instructions: `adds`, `subs`, and `cmp`

Standard instructions compute results without modifying flags unless explicitly requested:

The `s` Suffix Variant

- `adds x0, x1, x2`: Performs addition, stores sum in `x0`, and updates **NZCV** flags.
- `subs x3, x3, #1`: Subtracts and updates flags.

Comparison (`cmp`)

- `cmp x4, x5` is an alias for `subs xzr, x4, x5`.
- Subtracts operands to set flags, but **discards the result**.
- Feeds conditional branches like `b.eq` or `b.lt`.

Section 04

Multiplication, Division, and Shifts

Binary Multiplication Mechanics

- Multiplication operates as repeated shifted additions.
- Each multiplier bit selects a shifted copy of the multiplicand.
- Multiplying two n-bit values produces a full product up to $2n$ bits wide.
- Fixed-width registers typically retain only the lower n bits of the product.

Example (5×3)

$$0101_2 \times 0011_2$$

$$= 0101_2 + 1010_2 \text{ (shifted)}$$

$$= 1111_2 = 15_{10}$$

AArch64 Multiply and Divide Instructions

AArch64 provides dedicated hardware instructions for arithmetic expansion:

Instruction Syntax	Operation Performed	Architectural Effect
`mul x0, x1, x2`	64-Bit Multiplication	$x0 \leftarrow x1 \times x2$
`udiv x3, x4, x5`	Unsigned Division	$x3 \leftarrow x4/x5$ (unsigned)
`sdiv x3, x4, x5`	Signed Division	$x3 \leftarrow x4/x5$ (signed)

Microarchitectural Note

Multiplication and division instructions typically exhibit higher instruction latency than simple single-cycle additions.

Logical and Arithmetic Shift Operations

Shifts move bit patterns within a register to execute rapid scaling or isolation:

Logical Shift Left (`lsl`)

- Shifts bits left.
- Fills vacant low-order bits with zeros.
- Multiplies by powers of two (2^n).

Logical Shift Right (`lsr`)

- Shifts bits right.
- Fills vacant high-order bits with zeros.
- Unsigned division by 2^n.

Arithmetic Shift Right (`asr`)

- Shifts bits right.
- Preserves sign bit in high-order positions.
- Signed division by 2^n.

Section 05

Processor Performance: Clock Rate, CPI, and CPU Time

Processor Performance Basics

The Ultimate Metric: Execution Time

Performance is measured strictly by total program execution time, not by clock frequency alone.

- Processors execute ordered sequences of machine instructions.
- Each instruction takes one or more clock cycles depending on complexity and hazards.
- **CPI (Cycles Per Instruction)** summarizes the average execution cycles required per instruction across a workload.
- Instruction count, CPI, and clock rate must be considered together.

Clock Rate, Cycle Time, and CPI Formulas

Frequency and Cycle Time

- **Clock Rate**: Cycles per second (e.g., 2 GHz $= 2 \times 10^9$ cycles/s).
- **Cycle Time**: Duration of one clock cycle ($1/$Clock Rate).
- Example: 2 GHz $\rightarrow 0.5$ ns cycle time.

Cycles Per Instruction (CPI)

- Average clock cycles consumed per instruction.
- Varies based on instruction mix, memory latency, and microarchitectural pipelining.

The CPU Execution Time Equation

Fundamental Performance Equation

$$\text{CPU Time} = \text{Instruction Count} \times \text{CPI} \times \text{Clock Cycle Time}$$

$$\text{CPU Time} = \frac{\text{Instruction Count} \times \text{CPI}}{\text{Clock Rate}}$$

- A higher clock rate does not guarantee faster execution if CPI or instruction count increases disproportionately.
- **Worked Example**: $1,000,000$ instructions, CPI $= 2$, 1 GHz clock rate:

$$\text{CPU Time} = \frac{1,000,000 \times 2}{1,000,000,000} = 0.002 \text{ seconds} = 2 \text{ ms}$$

Section 06

Summary & Self-Test

Topic 4 Comprehensive Summary

- **Binary Addition & Subtraction**: Standard binary addition handles unsigned logic, while two's complement encoding allows the same adder to execute subtraction ($A + (-B)$).
- **Signed Overflow**: Occurs when results exceed container bounds, tracked independently from unsigned carries via the V flag.
- **Condition Flags**: Instructions with the `s` suffix and `cmp` update **NZCV** flags in PSTATE to drive conditional branching.
- **Arithmetic Extensions**: Multiplication (`mul`), division (`udiv`/`sdiv`), and shifts (`lsl`/`lsr`/`asr`) expand ALU capabilities.
- **Performance Equation**: CPU execution time is a function of Instruction Count, CPI, and Clock Rate.

Self-Test: Questions 1 & 2

Question 1: Addition and Carry

How does binary addition produce a carry, and what does the carry-out indicate for unsigned integer arithmetic?

Question 2: Two's Complement Subtraction

How can subtraction be performed using two's complement addition without requiring dedicated hardware subtraction circuitry?

Self-Test: Solutions 1 & 2

Solution 1

Binary addition generates a carry when the sum of bits in a column equals or exceeds 2, propagating a 1 into the next higher-order column. For unsigned arithmetic, a final carry-out indicates the result exceeded the representable range 0 to $2^n - 1$.

Solution 2

Subtraction $(A - B)$ is computed as $A + (-B)$. The negative operand $-B$ is formed by inverting all bits of B (one's complement) and adding 1, allowing the standard binary adder to process subtractions directly.

Self-Test: Questions 3 & 4

Question 3: Carry vs. Signed Overflow

What is the fundamental difference between carry (C) and signed overflow (V)? Can an operation have carry without overflow?

Question 4: Flag-Setting Instructions

What is the difference between `add` and `adds`, and how does `cmp` function as an alias for flag generation?

Self-Test: Solutions 3 & 4

Solution 3

Carry tracks unsigned range overflow (carry out of MSB), whereas signed overflow (V) tracks two's complement sign violations. Yes, adding large unsigned numbers can set $C = 1$ without violating signed bounds ($V = 0$).

Solution 4

`adds` computes the arithmetic sum and updates the NZCV condition flags in PSTATE, whereas standard `add` does not update flags. `cmp` subtracts operands to update flags for branching but discards the numeric result.

Self-Test: Questions 5 & 6

Question 5: Shifts and Powers of Two

What is the difference between `lsl`, `lsr`, and `asr`? When is each appropriate for multiplying or dividing by powers of two?

Question 6: CPU Performance Calculation

Consider two processors running the same 1-million instruction program:

- Processor A: CPI $= 2$, Clock Rate $= 2$ GHz
- Processor B: CPI $= 1$, Clock Rate $= 1$ GHz

Which processor has the lower execution time?

Self-Test: Solutions 5 & 6

Solution 5

`lsl` shifts left (multiplies by 2^n), `lsr` shifts right filling zeros (unsigned division by 2^n), and `asr` shifts right preserving the sign bit (signed division by 2^n).

Solution 6

- Processor A Time: $(10^6 \times 2)/(2 \times 10^9) = 0.001$ s (1 ms).
- Processor B Time: $(10^6 \times 1)/(1 \times 10^9) = 0.001$ s (1 ms).
- Both processors achieve identical execution time.

Wrap-Up and Next Steps

Moving Forward to Topic 5

Now that we have mastered integer arithmetic, condition flags, and basic performance metrics, our next session will explore:

- **Advanced AArch64 Addressing Modes**: Register-indirect, pre-index, and post-index memory loading.
- **Subroutine Linkage & Stack Frames**: How function calls, stack pointers (`sp`), and frame pointers (`x29`) manage local storage.

End of Topic 4 — Questions & Discussion